

Failure Cases

Where our system can break or perform poorly, why, and what we do about it. Figures marked "sim" come from our PyBullet runs (default seed).

1. SLAM heading drift in long, featureless corridors

- **Limitation:** stock BreezySLAM (RMHC scan matching) drifts in heading along long straight corridors, about $-0.2^\circ/\text{m}$ measured. Turning in place makes it worse: -36° over 18 s of turning.
- **Why:** a perfectly featureless straight tunnel gives scan matching nothing to hold on to along the corridor.
- **How we handle it:** damped fusion of scan matching with odometry and gyro, no scan-match correction while rotating fast, and a wall-direction heading reference. We also added wall recesses to the simulated tunnels. Result (sim): mean position error 0.14 m, max 0.28 m, while odometry alone ends 4-8 m off (Figure 1).
- **What remains:** the wall-direction reference assumes an orthogonal, room-and-pillar mine layout. For irregular tunnels we would turn it off and expect gyro-bias heading drift. The map is also centred on the start pose, so the mine must lie within ± 40 m of it.

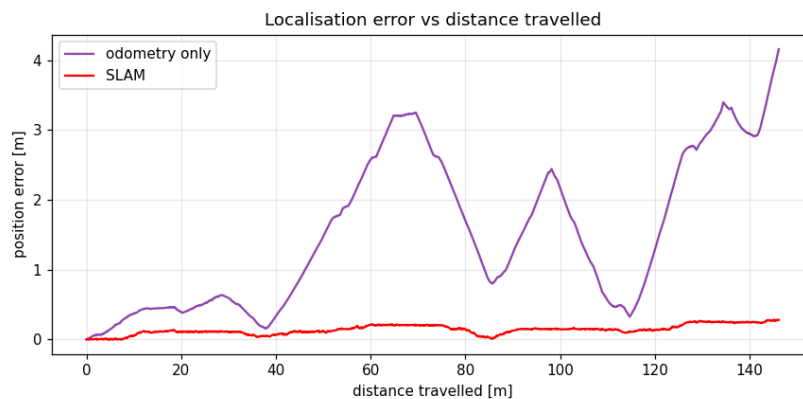


Figure 1. Position error vs distance travelled: SLAM (red) against odometry only (purple).

2. Sensor false positives and false negatives

- **MQ-2 gas:** it is not selective for toxic gases and is affected by temperature and humidity. Dust, exhaust or other airborne chemicals can raise the reading and cause a false alarm. We use it as a threshold-above-baseline gas/smoke anomaly trigger, not a calibrated measurement. The long preheat and baseline drift also matter. Mitigation: trigger only on a sustained rise above the warmed-up baseline, and give gas MEDIUM priority.
- **PIR motion (trapped worker):** it can false-trigger on warm equipment or moving air. It can also miss a worker who lies perfectly still, because it detects warm bodies that *move*. The robot's own movement changes what the PIR sees. Mitigation: ignore triggers during the 30-60 s warm-up, read only while the robot is stopped, and require repeat triggers. A missed still worker is a real risk we cannot remove with this sensor.
- **Tilt/vibration switch (collapse):** the ball bounces and also trips on the robot's own bumps. It cannot tell how big an event was. Mitigation: debounce in firmware, require a few triggers inside a short window, and tune the sensitivity potentiometer on the real chassis.
- **What our simulation does not test:** simulated events fire when the robot's true position enters a predefined zone, so false positives and negatives are never exercised. Bench tests in Phase 2 are where these get measured.

3. RF range limits

- **Limitation:** nRF24L01 radios have limited range, and rock and metal structures underground cut it further. If the Writer gets too far from the outside network node, beacons can be lost.
- **Why it matters:** this limits how large a tunnel network we can cover with a single node.
- **How we handle it:** beacons carry a priority-based TTL and rebroadcast interval (high priority: TTL 20, every 2 s), so urgent findings get more chances to get through. That does not help once the Writer is fully out of range.
- **Known direction (not built):** chain several relay nodes as a mesh, and use PA+LNA radio modules if range tests demand it.
- **What our simulation does not test:** radio delivery is a direct function call in simulation, so packet loss is not modelled. The Phase 2 radio test measures packet loss against distance.

4. Frame translation assumptions

- **Limitation:** the GPS conversion uses a flat-earth approximation and a one-time manual calibration at the tunnel entrance (GPS position and compass bearing of the robot's initial heading).
- **Why:** the approximation is fine at the scale of one tunnel network, but accuracy would degrade over much larger distances. A wrong calibration reading shifts every beacon consistently. A heading error of 2° moves a beacon 100 m from the entrance by about 3.5 m, and 5° by about 8.7 m. A phone GPS fix error moves every beacon by the same offset.
- **How we handle it:** we chose manual calibration over an onboard magnetometer, since compasses are unreliable near metal and mineral content. The Executor uses beacon coordinates in the shared entrance frame, so GPS calibration errors affect the command-post map but not the Executor's driving. Placing both robots at the same marked entrance pose matters, and the Phase 2 plan measures it.

5. Executor navigation is not yet reliable

- **Limitation:** the Executor's goal-seeking controller is the least mature part of the system. We ran the repo's integration demo (`run_integration_demo_v2.py`, unmodified, one run, default seed). The Writer finished normally, but the Executor reached **1 of 3** beacon targets and stopped at the 900 s time limit.
- **Why:** the controller steers straight toward the target and, when something is ahead, rotates in place until the way clears. It has no path planning, so it can stall behind walls. In the run it spent most of the time in its AVOIDING state from about $t=60$ s, covering only about 40 m in the remaining 840 s. Its position error also grew to 1.88 m, against a Writer maximum of 0.28 m, consistent with the turning-in-place SLAM drift in case 1. Steering gains are reused from the Writer's wall-following controller and have not been tuned for goal-seeking.
- **How we plan to handle it:** plan paths over the SLAM occupancy map (for example A* on `nav.get_map()`), fall back to wall-following when AVOIDING lasts too long, and retune the gains. These are not implemented yet.